

day19笔记

一、SequentialChain

- 1、SequentialChain 顺序链，可以把多个链连接起来
- 2、会把第一个链的结果返回给定第二个链，依此类推...
- 3、SimpleSequentialChain简单顺序链

不使用顺序链

```
from langchain_classic.chains.llm import LLMChain
from langchain_community.chat_models.tongyi import ChatTongyi
from langchain_core.prompts import PromptTemplate, ChatPromptTemplate
from dotenv import load_dotenv
load_dotenv()

llm = ChatTongyi(model='qwen3-max')

# 任务
# 1、生成一个故事标题
prompt1 = PromptTemplate.from_template('给我一个关于{topic}的故事标题')
# 2、根据标题生成故事
prompt2 = PromptTemplate.from_template('根据{title}写一个200字左右的故事')
# 3、总结故事寓意
prompt3 = PromptTemplate.from_template('使用一句话总结这个故事的寓意：{story}')

# 定义三个链
chain1 = LLMChain(llm=llm, prompt=prompt1)
chain2 = LLMChain(llm=llm, prompt=prompt2)
chain3 = LLMChain(llm=llm, prompt=prompt3)

title = chain1.run('人工智能')
print('生成标题', title)

print('-' * 30)

story = chain2.run(title)
print('生成故事', story)

print('-' * 30)

moral = chain3.run(story)
print('生成寓意', moral)
```

使用顺序链

```
from langchain_classic.chains.llm import LLMChain
from langchain_classic.chains.sequential import SimpleSequentialChain
from langchain_community.chat_models.tongyi import ChatTongyi
from langchain_core.prompts import PromptTemplate, ChatPromptTemplate
```

```

from dotenv import load_dotenv
load_dotenv()

llm = ChatTongyi(model='qwen3-max')

# 任务
# 1、生成一个故事标题
prompt1 = PromptTemplate.from_template('给我一个关于{topic}的故事标题')
# 2、根据标题生成故事
prompt2 = PromptTemplate.from_template('根据{title}写一个200字左右的故事')
# 3、总结故事寓意
prompt3 = PromptTemplate.from_template('使用一句话总结这个故事的寓意：{story}')

# 定义三个链
chain1 = LLMChain(llm=llm, prompt=prompt1)
chain2 = LLMChain(llm=llm, prompt=prompt2)
chain3 = LLMChain(llm=llm, prompt=prompt3)

# 创建顺序链
chains_resp = SimpleSequentialChain(
    chains=[chain1, chain2, chain3],
    verbose=True
)

result = chains_resp.invoke('人工智能')
print(result)

```

4、SequentialChain高级顺序链

```

from langchain_classic.chains.llm import LLMChain
from langchain_classic.chains.sequential import SimpleSequentialChain, SequentialChain
from langchain_community.chat_models.tongyi import ChatTongyi
from langchain_core.prompts import PromptTemplate, ChatPromptTemplate
from dotenv import load_dotenv
load_dotenv()

llm = ChatTongyi(model='qwen3-max')

# 文章创作流水线
# 1、生成文章标题
# output_key='title' 指定输出的变量名
title_prompt = PromptTemplate.from_template('为关于{topic}的文章生成一个吸引人的标题')
title_chain = LLMChain(
    llm=llm,
    prompt=title_prompt,
    output_key='title'
)

# 2、生成文章大纲
outline_prompt = PromptTemplate.from_template('根据{title}，生成一个文章大纲（3-5个要点）')
outline_chain = LLMChain(
    llm=llm,
    prompt=outline_prompt,

```

```

        output_key='outline'
    )

# 3、撰写文章
article_prompt = PromptTemplate.from_template('根据{title}和{outline}, 撰写500字左右的文章')
article_chain = LLMChain(
    llm=llm,
    prompt=article_prompt,
    output_key='article'
)

# 4、生成摘要
summary_prompt = PromptTemplate.from_template('为以下文章生成一个50字左右的摘要: \n{article}')
summary_chain = LLMChain(
    llm=llm,
    prompt=summary_prompt,
    output_key='summary'
)

# 创建高级顺序链
# output_variables 表示把中间的结果都返回给你
chain_resp = SequentialChain(
    chains=[title_chain, outline_chain, article_chain, summary_chain],
    input_variables=['topic'],
    output_variables=['title', 'outline', 'article', 'summary'],
    verbose=True
)

result = chain_resp.invoke({'topic': 'AI在教育中的应用方向'})
print(f"标题: \n {result['title']}")
print(f"大纲: \n {result['outline']}")
print(f"文章: \n {result['article']}")
print(f"摘要: \n {result['summary']}")

```

二、Memory记忆介绍

1、大模型本身是没有记忆功能的，需要自己去添加记忆，之前的做法是把每次聊天的内容一起发送给大模型

示例代码

```

from langchain_community.chat_models.tongyi import ChatTongyi
from langchain_core.prompts import PromptTemplate, ChatPromptTemplate
from dotenv import load_dotenv
load_dotenv()

llm = ChatTongyi(model='qwen3-max')

result1 = llm.invoke('你好，我是张三，我是一名python开发者').content
print(result1)

print('-' * 30)

result2 = llm.invoke('我的名字是什么? ').content

```

```
print(result2)
```

输出结果

你好，张三！很高兴认识你。作为一名 **python** 开发者，你一定对编程充满热情。如果你有任何关于 **python** 的问题、需要代码帮助、项目建议，或者想讨论某个技术话题（比如 **web** 开发、数据分析、机器学习、自动化脚本等），随时告诉我！

你现在在做什么项目呢？或者有什么我可以帮你的吗？😊

我还不知道你的名字呢！你可以告诉我我希望我怎么称呼你吗？😊

2、为了解决模型没有记忆的能力，langchain给咱们提供了专门的模块来解决

3、langchain里面提供的记忆分为短期记忆和长期记忆

三、短期记忆

```
from langchain_community.chat_models.tongyi import ChatTongyi
# 记忆相关模块
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.runnables.history import RunnableWithMessageHistory
from langchain_core.chat_history import InMemoryChatMessageHistory

from dotenv import load_dotenv
load_dotenv()

llm = ChatTongyi(model='qwen3-max')

# 创建提示词模版
# MessagesPlaceholder() 是占位符，是专门用来插入历史对话的
# variable_name='history' 表示在输入的字典中，历史消息会放在这个键中
# input 用户问题的输入最终会放在这个键里面 invoke({'input': ''})
prompt = ChatPromptTemplate.from_messages([
    ('system', '你是一个乐于助人的AI助手，问题请使用中文回答我'),
    MessagesPlaceholder(variable_name='history'),
    ('human', '{input}')]
])

# 组合可执行链
chain = prompt | llm

# 创建存储所有历史消息的字典
store = {}

# 设置一个可以获取历史记录函数
# session_id 会话的唯一标识（可以区分不同的窗口或者不同的用户，可以做到会话隔离）
# InMemoryChatMessageHistory() 内存盒子（存储记忆的容器）
def get_session_history(session_id: str):
    if session_id not in store:
        store[session_id] = InMemoryChatMessageHistory()

    return store[session_id]
```

```

# RunnableWithMessageHistory 使用包装链，这个是整个记忆的核心，会在调用前自动处理历史消息
# chain 要进行包装的原始链
# get_session_history 获取历史记录的函数
# input_messages_key 告诉包装器，用户的输入在字典中那个键里面
# history_messages_key 告诉包装器，历史消息在字典的那个键里面
with_history = RunnableWithMessageHistory(
    chain,
    get_session_history,
    input_messages_key='input',
    history_messages_key='history'
)

# 创建会话配置
config = {'configurable': {'session_id': 'user_A'}}

print('第一轮对话', '-' * 30)
resp1 = with_history.invoke({'input': '你好，我叫小明，是一名python开发工程师'}, config)
print(resp1.content)

print('-' * 30)

resp2 = with_history.invoke({'input': '我的名字是什么?'}, config=config)
print(resp2.content)

print('第二轮对话', '-' * 30)

config2 = {'configurable': {'session_id': 'user_B'}}

resp3 = with_history.invoke({'input': '你好，我叫俞成儒，是一名java开发工程师'}, config=config2)
print(resp3.content)

print('-' * 30)

resp4 = with_history.invoke({'input': '我叫什么名字，我的工作是什么?'}, config=config2)
print(resp4.content)

print('-' * 50)
history_box = get_session_history(session_id='user_A')
print(history_box)

```

四、滑动窗口记忆

- 1、所谓的滑动窗口记忆指的是记忆只能记住几轮，可以规定只记住三轮的记忆
- 2、示例代码

```

from langchain_community.chat_models.tongyi import ChatTongyi
# 记忆相关模块
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.runnables.history import RunnableWithMessageHistory
from langchain_core.chat_history import InMemoryChatMessageHistory

```

```

from dotenv import load_dotenv
load_dotenv()

llm = ChatTongyi(model='qwen3-max')

# 创建提示词模版
# MessagesPlaceholder(variable_name='history') 起了一个占位作用，用于插入历史消息
prompt = ChatPromptTemplate.from_messages([
    ('system', '你是一个乐于助人的AI助手，请使用中文回答我问题'),
    MessagesPlaceholder(variable_name='history'),
    ('human', '{input}')]
])

# 组装链
chain = prompt | llm

# 创建存储的字典 (story)
story = {}

# 创能获取几轮的记忆函数 k
def get_history_and_invoke(user_input, session_id, keep_turns=2):
    if session_id not in story:
        story[session_id] = InMemoryChatMessageHistory()

    # 获取完整的原始历史记录列表
    # messages 是InMemoryChatMessageHistory()这个里面的属性，专门存储历史消息
    full_history = story[session_id].messages

    # 进行历史消息的截取
    # max_messages 表示从哪里开始截取消息的索引
    # recent_history 表示的是截取后的历史消息
    max_messages = keep_turns * 2
    recent_history = full_history[-max_messages:]

    # 调用模型
    responses = chain.invoke({
        'history': recent_history,
        'input': user_input
    })

    # 更新存储
    # add_user_message(user_input) 表示添加的是用户输入的消息
    story[session_id].add_user_message(user_input)
    # .add_ai_messages(responses.content) 表示添加的是AI回复的消息
    story[session_id].add_ai_message(responses.content)

    # 把最终结果返回给外界
    return responses

session_id = 'user_A'

```

```
print('=' * 30)
```

注意点：滑动窗口记忆，是做了记忆截断。但是你如果后面问的问题是相同的会把ai回答重新添加进记忆中，所以出现了你感觉都记住了的情况

```
result1 = get_history_and_invoke('我叫丁致宇，是一名大数据工程师', session_id, keep_turns=2)
print(result1.content)
```

```
print('-' * 30)
```

```
result2 = get_history_and_invoke('我喜欢坤坤', session_id, keep_turns=2)
print(result2.content)
```

```
print('-' * 30)
```

```
result3 = get_history_and_invoke('我喜欢rap、唱歌、跳舞', session_id, keep_turns=2)
print(result3.content)
```

```
print('-' * 30)
```

```
result4 = get_history_and_invoke('我还喜欢喝咖啡', session_id, keep_turns=2)
print(result4.content)
```

```
print('-' * 30)
```

```
result5 = get_history_and_invoke('我的名字是什么?', session_id, keep_turns=2)
print(result5.content)
```

五、长期记忆

- 1、短期记忆，当程序重启动后，那么之前的记忆就不存在了
- 2、长期记忆，可以给记忆做持久化，例如：可以写入数据库、也可以写入一个json文件中...
- 3、这里采用的方案是写入json文件中，通过 session_id做隔离
- 4、示例代码

```
import os
```

```
from langchain_community.chat_message_histories import FileChatMessageHistory
from langchain_community.chat_models.tongyi import ChatTongyi
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.runnables.history import RunnableWithMessageHistory
from langchain_core.chat_history import InMemoryChatMessageHistory
```

```
from dotenv import load_dotenv
load_dotenv()
```

```
llm = ChatTongyi(model='qwen3-max')
```

```
# 创建提示词模版
```

```
prompt = ChatPromptTemplate.from_messages([
```

```

    MessagesPlaceholder(variable_name='history'),
    ('human', '{input}'))
])

# 组装链
chain = prompt | llm

# 定义json文件存储目录
HISTORY_DIR = './my_chat_history'

# os.makedirs() 可以检测咱们需要创建的目录是否存在，如果没有就创建，有就直接使用了。如果不做验证，那么执行
# 的时候找不到这个目录就会报错
os.makedirs(HISTORY_DIR, exist_ok=True)

# 创建获取历史记录的函数
def get_file_history(session_id: str):
    # 现在前面已经有目录了，那么需要给创建一个json文件
    # 对路径的处理，输出结果: my_chat_history/user_A.json
    file_path = os.path.join(HISTORY_DIR, f'{session_id}.json')

    # FileChatMessageHistory(file_path) 会自动处理文件的写入和读取。如果文件存在就写入，文件不存在就先
    # 创建再写入（最终返回的是一个文件读写对象）
    return FileChatMessageHistory(file_path)

# 包装链
with_file_history = RunnableWithMessageHistory(
    chain,
    get_file_history,
    history_messages_key='history',
    input_messages_key='input'
)

config = {'configurable': {'session_id': 'user_A'}}

# result1 = with_file_history.invoke({'input': '我叫丁致宇，是一名python开发工程师'},
# config=config)
# print(result1.content)

print('-' * 30)

result2 = with_file_history.invoke({'input': '我的名字是什么? '}, config=config)
print(result2.content)

```

5、作业

- 把今天所有知识点和案例过两遍
- 做一个带有长期记忆和短期记忆的聊天机器人，要有界面